

Rent Impromptu

2. Delegation

- Charles
 - Test Plan
- Aidan
 - Comparisons with similar designs
- Lana
 - Conclusion
- Joyce
 - Presentation slides
 - Transfer Deliverable #1 Content
 - Submission
- Abdallah
 - Project scheduling
- Vinh
 - Cost, Effort and pricing estimation.

3. Deliverable #1 Content

GitHub Repository: <https://github.com/RustyReference/3354-team6>

General Assumptions:

- Users should have their IDs verified before being able to access the platform for safety purposes.

Software Process Model Used: Incremental Software Process

The incremental software process combines properties of both linear and parallel process flows. Each linear sequence is typically as follows: communication, planning, modeling, construction, and deployment of increment. Meanwhile, other increments are also available to start on, bringing out the parallel process. It is noted that an increment can use the prototyping paradigm.

Advantages of this software process include reduced costs in accommodating for users' changing requirements, ease of getting user feedback on work that has been completed, and that delivery and deployment of parts of the software to the user that may prove to be useful is sped up and feasible.

The incremental software process is suitable for our project because this application should be user focused. We should cater features and interfaces in favor of the application's users. Additionally, we should accept user feedback and try to incorporate it

into the application to improve the app for users. This software process model is especially appealing for this purpose.

It would also be beneficial to push out parts of the software which may improve the user experience even if the entire program is not complete. Additionally, when new features are added or updates are necessary, this is equivalent to adding and implementing an increment.

Functional Requirements

1. *User Account Management*

- a. Registration: Users must be able to create an account using an email, username, and password. The user shall also be able to save their registration information through the initial login.
- b. Authentication: The system will institute a multi-factor verification method before granting access to account-related features.
- c. Profile Management: Users will be able to update their profile information like their profile picture, contact information, address, etc.
- d. Account Deletion: The software will allow users to delete their account, removing and anonymizing their personal data.
- e. Role Management: There shall be two main roles, the renter who can borrow items, and the owner rents out items. A user may be able to register as both.

2. *Item Listing Management*

- a. Create Listing: Users will be able to add new items with a title, description, category, condition, photos, and price.
- b. Update Listing: Owners will be able to modify the listing details, such as price or availability.
- c. Delete Listing: Owners will also be able to remove items no longer available to rent.
- d. View Listings: Users can view detailed information about each item like images, owner information, and reviews. Users will be able to interact with these listings, and ask questions about listings. Users will also be able to access the owner's contact information through the listing.

3. *Item Search and Filtering*

- a. Search Functionality: Users will be able to search for items using keywords, categories, or location.
- b. Filtering and Sorting: Search results can be filtered by price range, distance, category, condition, and rating. Sorting options will also include options like most recent, lowest price, and highest rated.

4. *Renting and Reservations*

- a. Booking System: Users will be able to select a date range and request to rent an item.
 - b. Availability Checking: The system shall verify if the item is available for the requested dates before confirming.
 - c. Owner Approval: The owner can either approve or deny rental requests in the allotted time.
 - d. Return Confirmation: Once the rental period ends, both parties shall confirm that the item has been returned.
5. *Payments and Deposits*
- a. Online Payment Integration: System is integrated with a secure payment gateway like Paypal, ApplePay, etc.
 - b. Deposit Handling: Deposits are handled temporarily and refunded after rental ends if no issues occur.
 - c. Transaction History: Users will be able to view past payments, refunds, and current rental transactions.
 - d. Automated Invoices: The system will generate receipts for every transaction that will be sent via the user's email. Both parties will get confirmation once the payment is processed.
6. *Reviews and Ratings*
- a. Post-rental Reviews: Both the renter and owner will be able to leave a review and rating after the rental period ends.
 - b. Average Rating Display: Each item and user profile will show an average rating for transparency.
7. *Customer Support*
- a. Help Center: There will be a dedicated section that provides FAQs, guidelines, and support documentation.
 - b. Chat/Message System: Users can message computer service or submit tickets for issues.
 - c. Automated Notifications: System will send notifications for booking confirmations, upcoming return dates, payment receipts, etc.

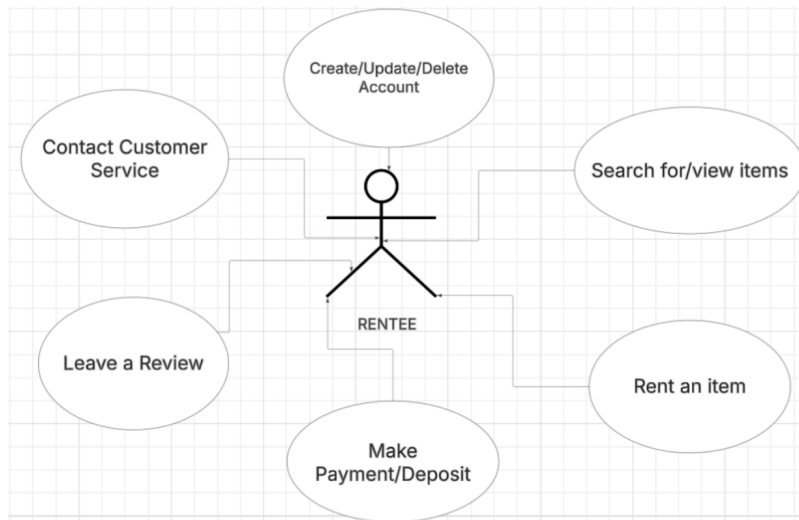
Non-Functional Requirements

1. *Performance*: The system should be able to handle multiple concurrent users without noticeable delays. Search results and item lists should take less than two seconds to load. Database queries should be optimized for adequate scalability.
2. *Security*: All sensitive data like passwords and payment information must be incremented using methods like encrypt. The system should use HTTPS for secure communication and role-based access must ensure users can access only their side of data.

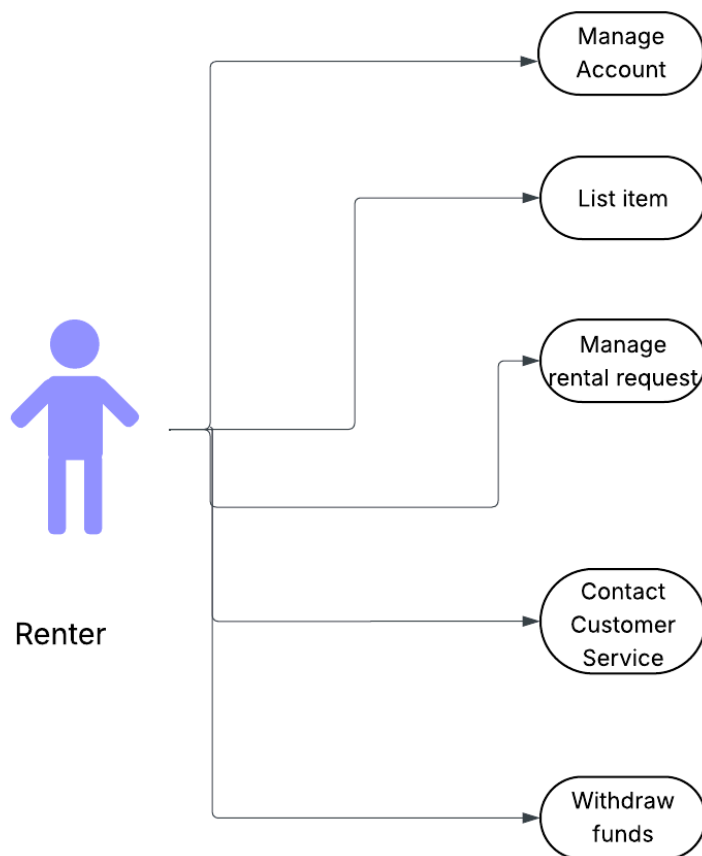
3. *Usability*: The user interface should be intuitive and accessible on both desktop and mobile devices. There should be minimal steps to list or rent an item, and support documentation should be easy to read and apply.
4. *Dependability*: Data integrity must be maintained during transactions. There should not be any loss of user or payment information during failures. The system shall have a minimum uptime of 99% even during scheduled maintenance.
5. *Scalability*: The system should be able to scale horizontally to handle the growth in the number of users, listings, and transactions. The design shall allow the integration of additional services without major reconstructing of the platform.
6. *Maintainability*: The code base shall be modular with well written documentation to simplify debugging and scaling efforts.
7. *Compatibility*: The web application should adapt responsibly to different operating systems and screen sizes and it should be compatible with major browsers. The system should support integration with standard payment APIs.
8. *Space*: The system must efficiently manage storage and bandwidth and uploaded media files should be compressed while maintaining acceptable quality.
9. *Operational*: The platform should remain functional during updates, peak usage periods, and small system failures. Administrators should have access to tools for troubleshooting.
10. *Developmental*: Development shall follow version control practices and include testing at performance levels like in unit, integration, and user acceptance.
11. *Ethical*: System should comply with all applicable privacy and data protection laws. User data will not be shared with third parties without consent.
12. *Accounting*: All financial transactions will be securely logged for audit purposes. Reports shall record all transactions for both users and administrators.
13. *Safety*: The platform must ensure physical and digital safety users during transactions. Guidelines shall be provided to renters and owners for safe item exchanges. The system shall provide safety alerts and allow administrators to remove users violating policies.
14. *Regulatory*: The system shall comply with national regulations governing online transactions and data protection.

Use Case Diagrams

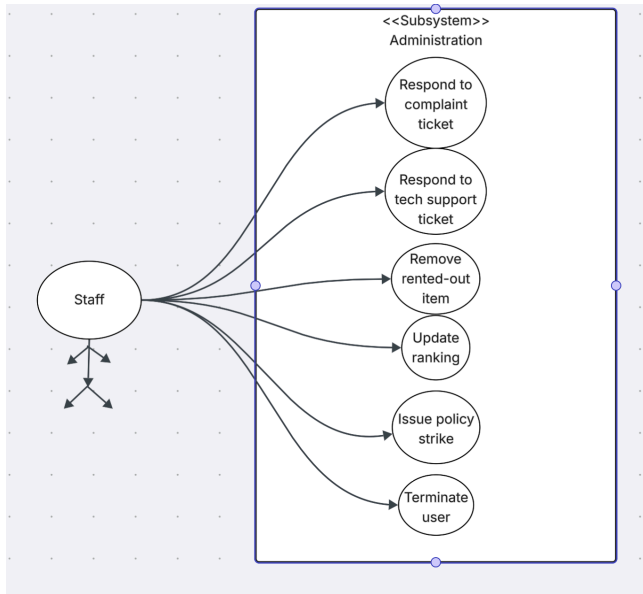
Rentee Diagram



Renter Diagram

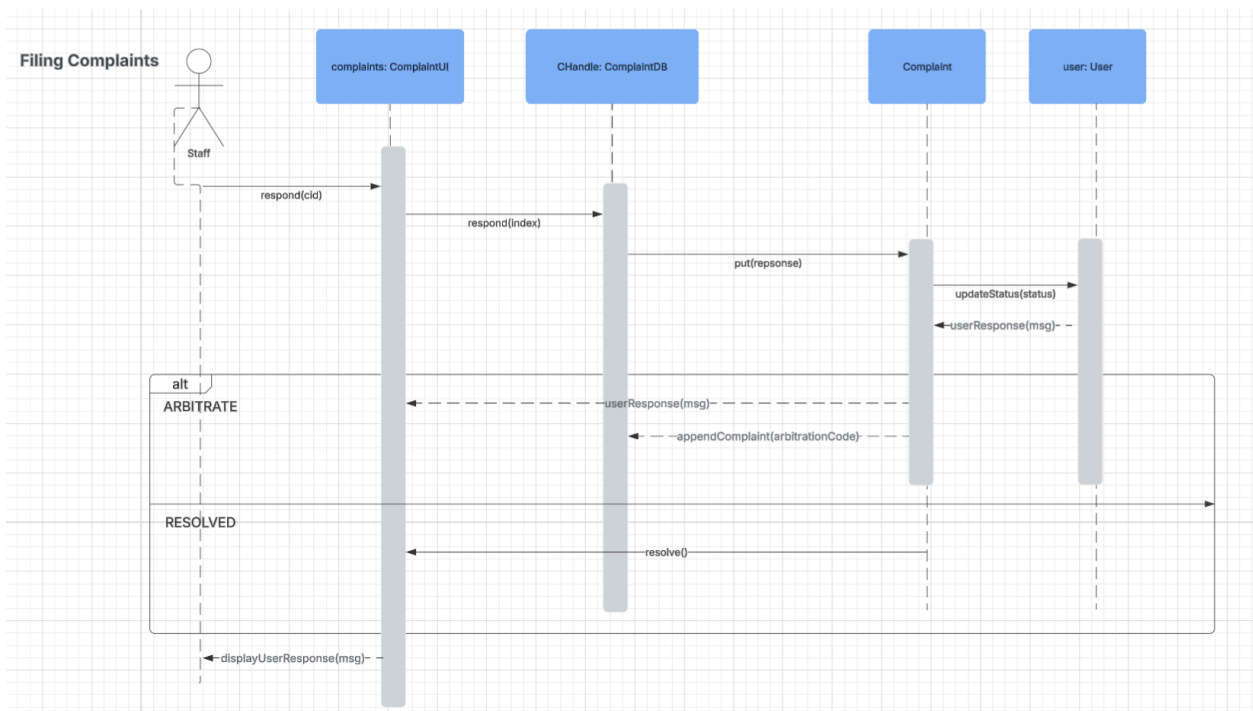


Staff Diagram

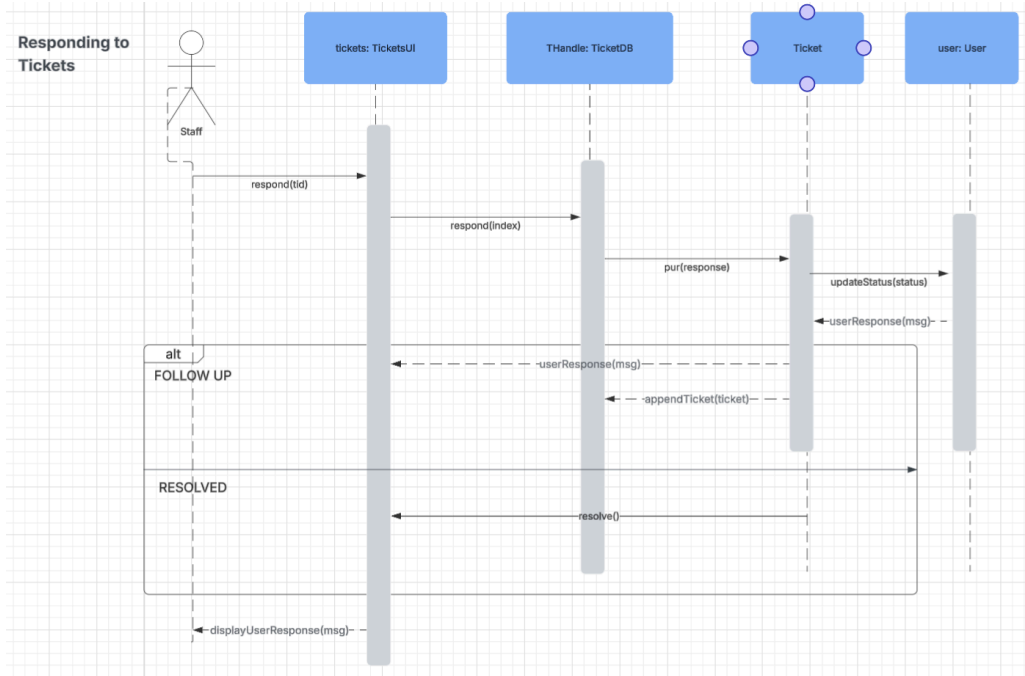


Sequence Diagrams

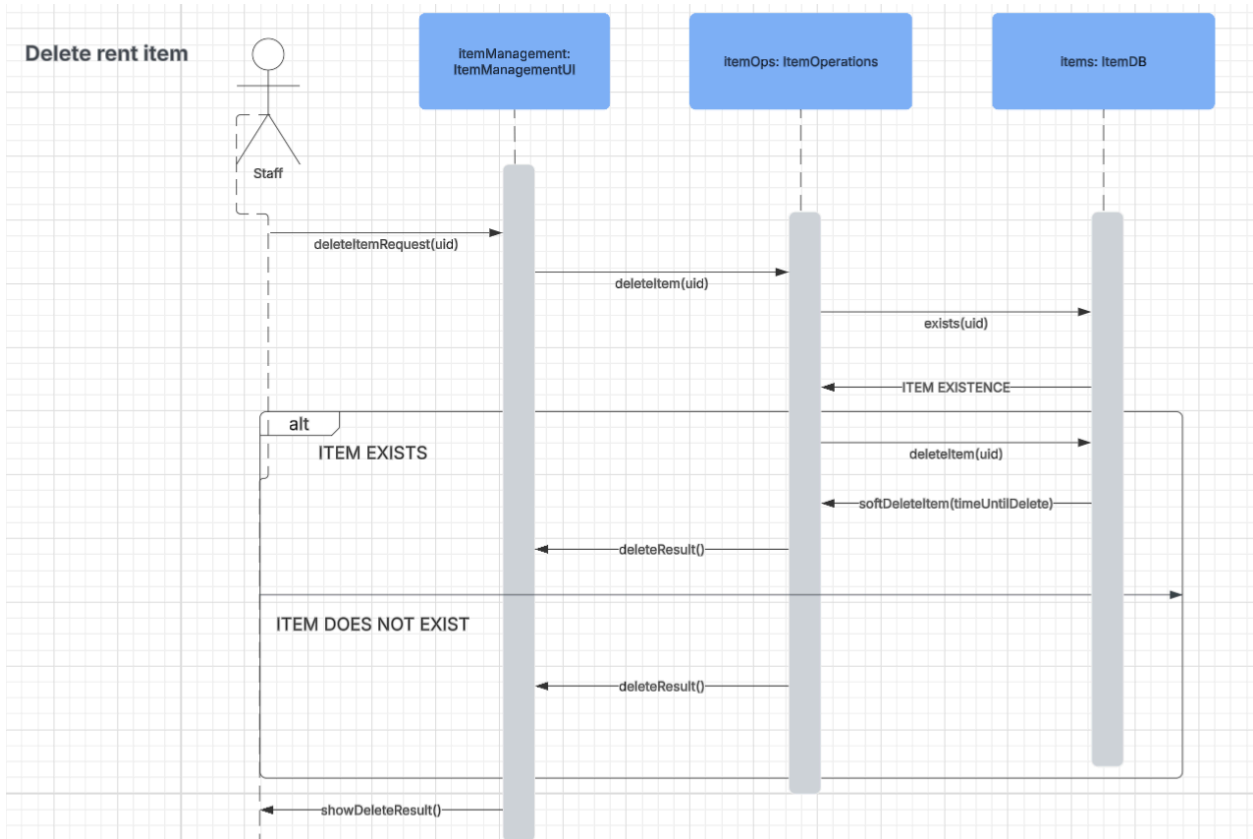
Staff: Filing Complaints



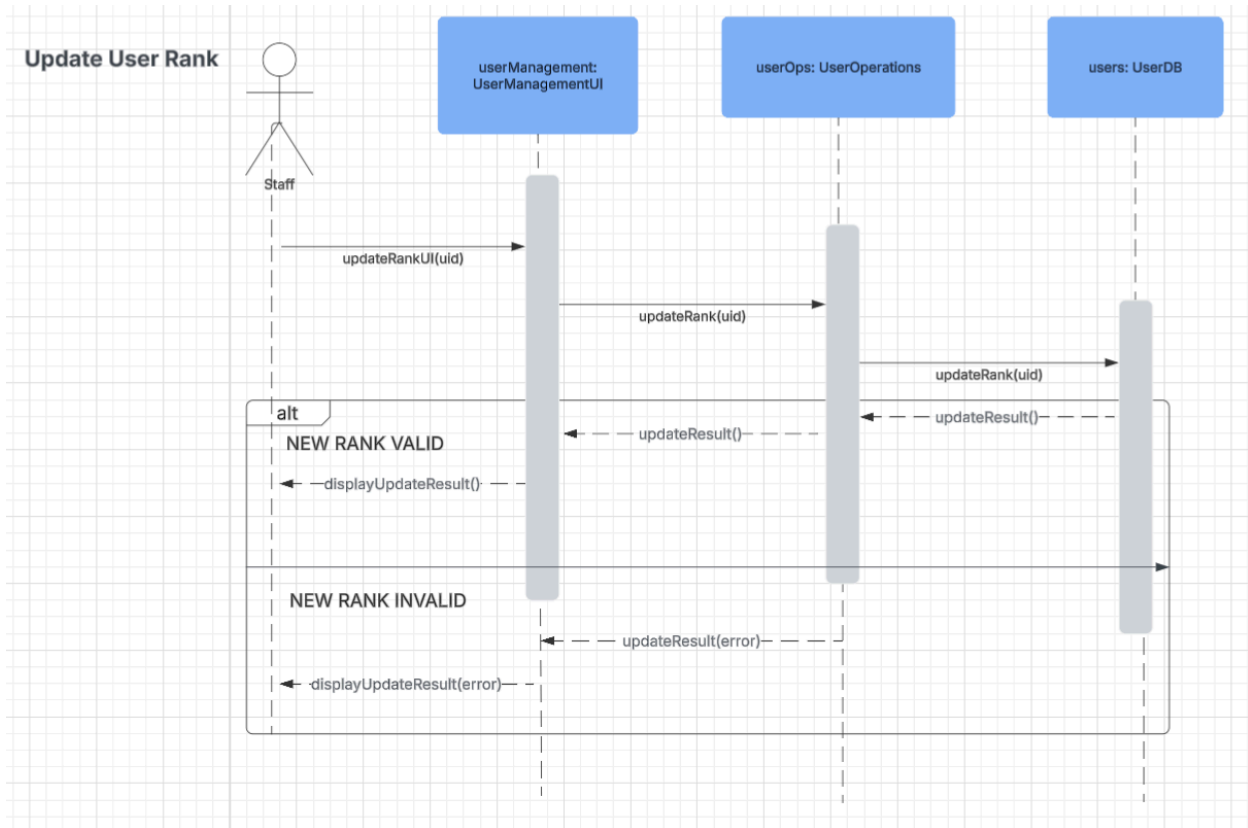
Staff: Responding to Tickets



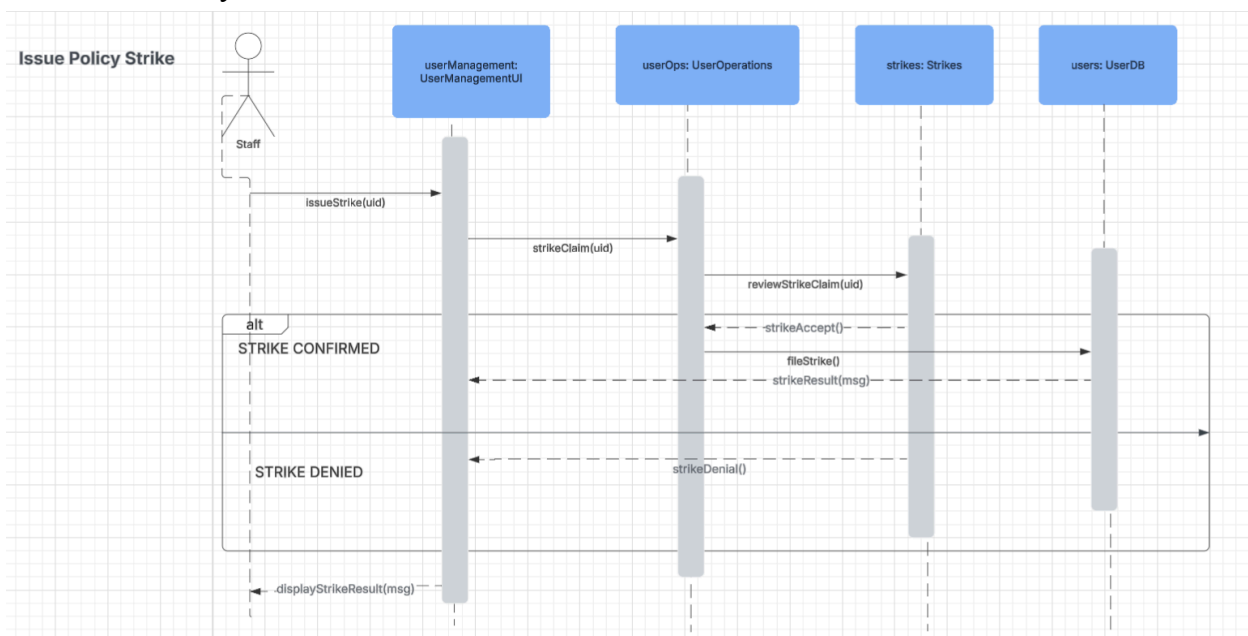
Staff: Delete Rent Item



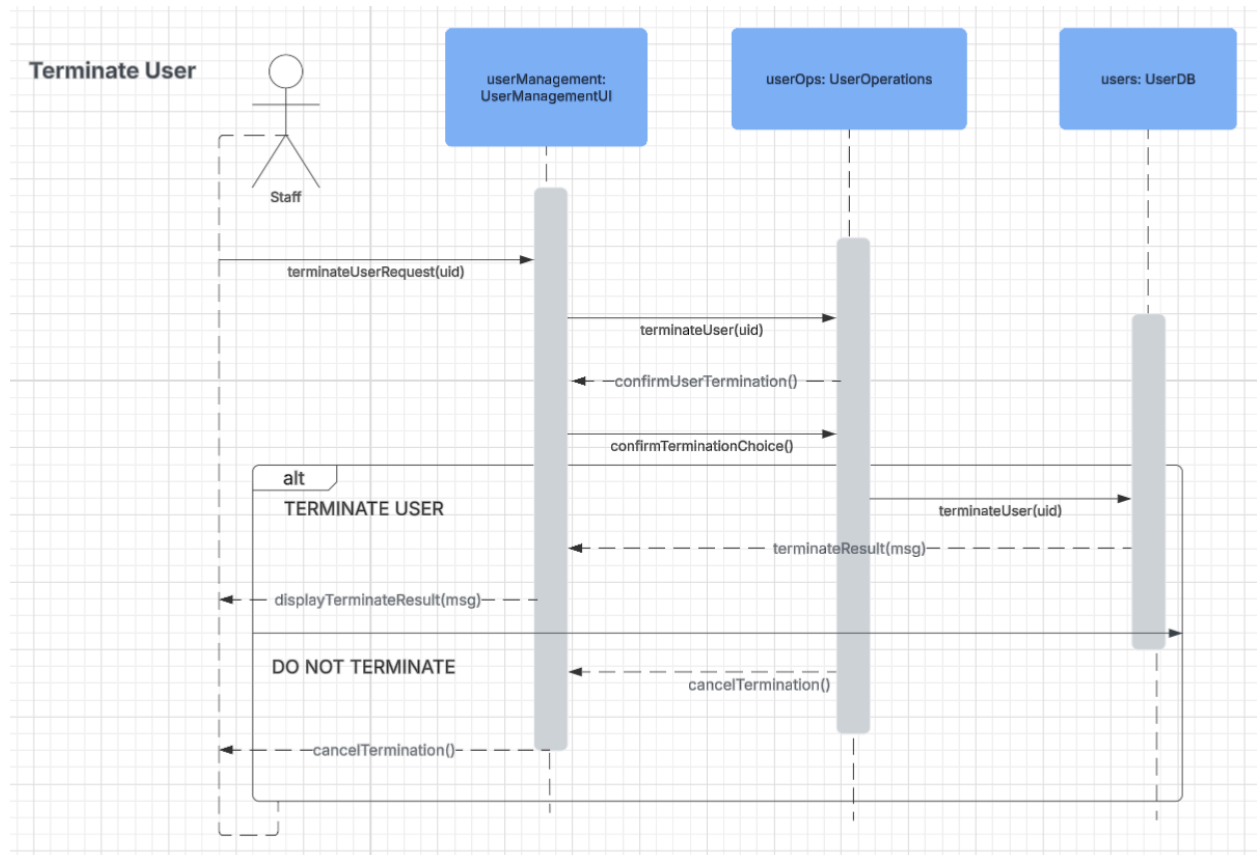
Staff: Update User Rank



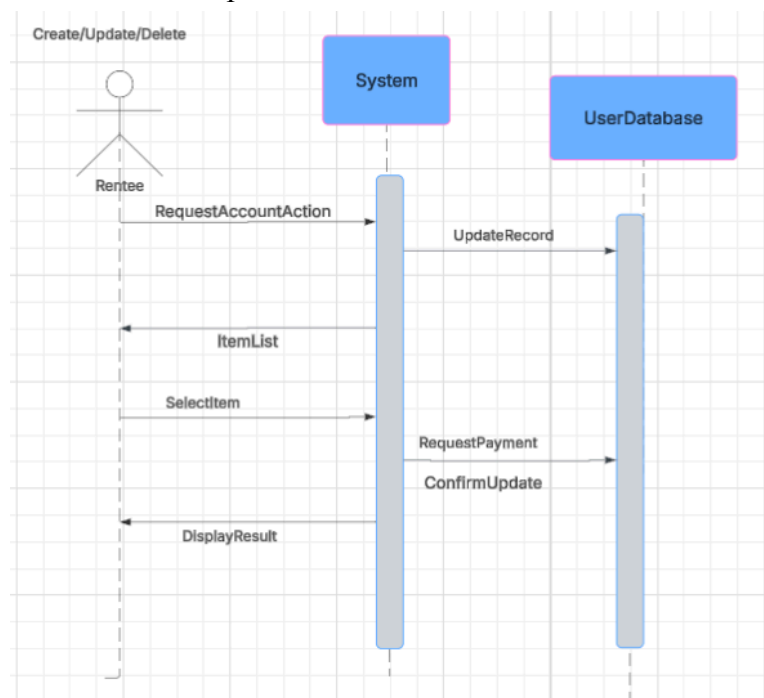
Staff: Issue Policy Strike



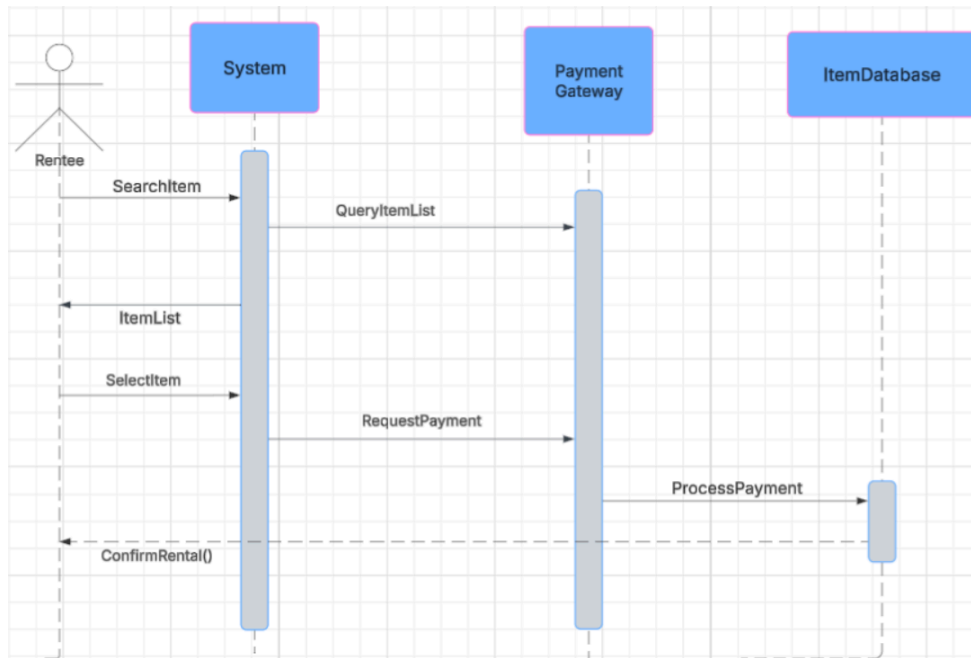
Staff: Terminate User



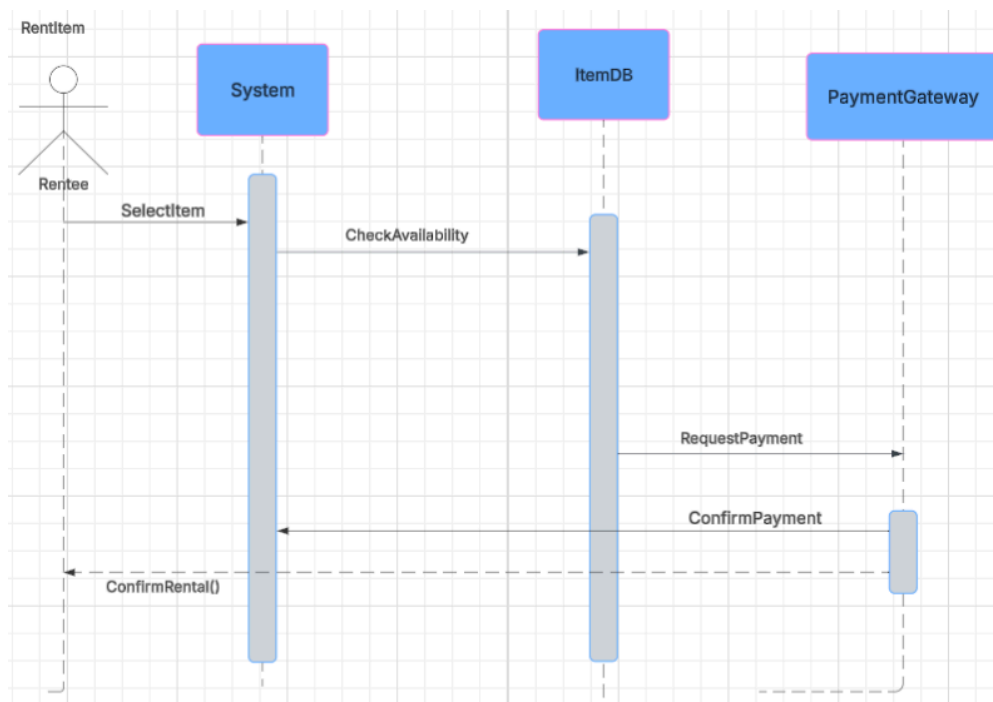
Rentee: Create/Update/Delete



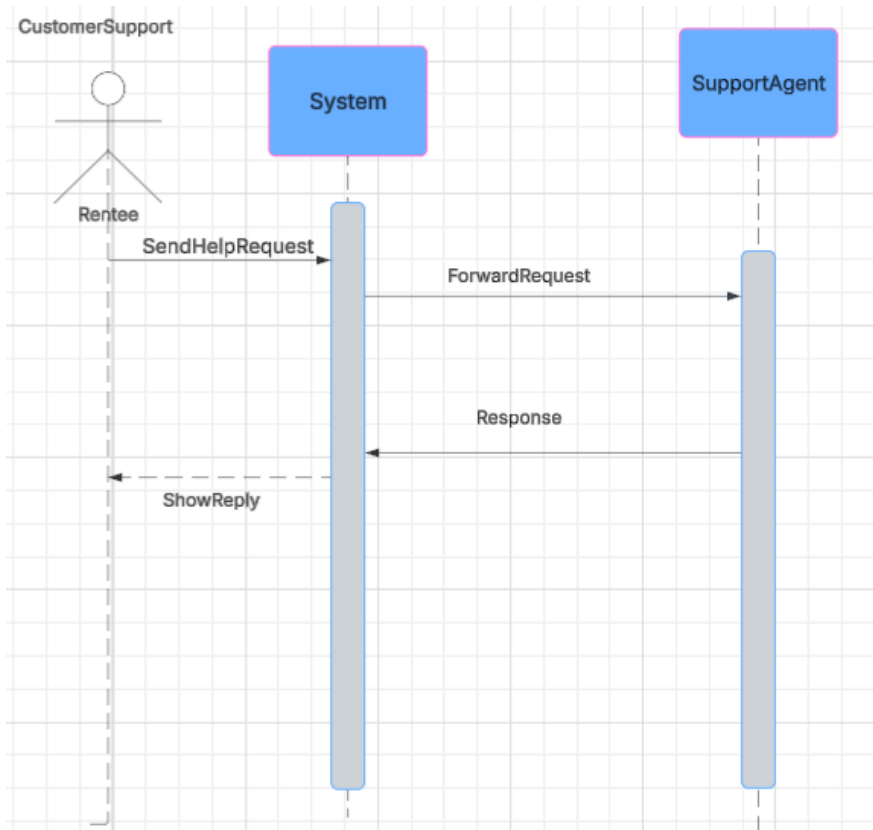
Rentee: Search for Items/Make Payments



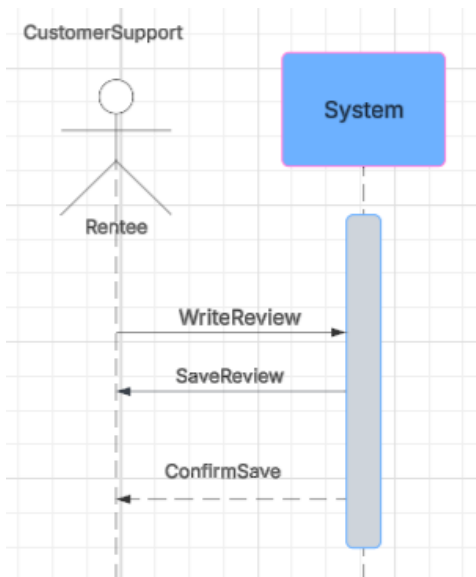
Rentee: Rent an Item



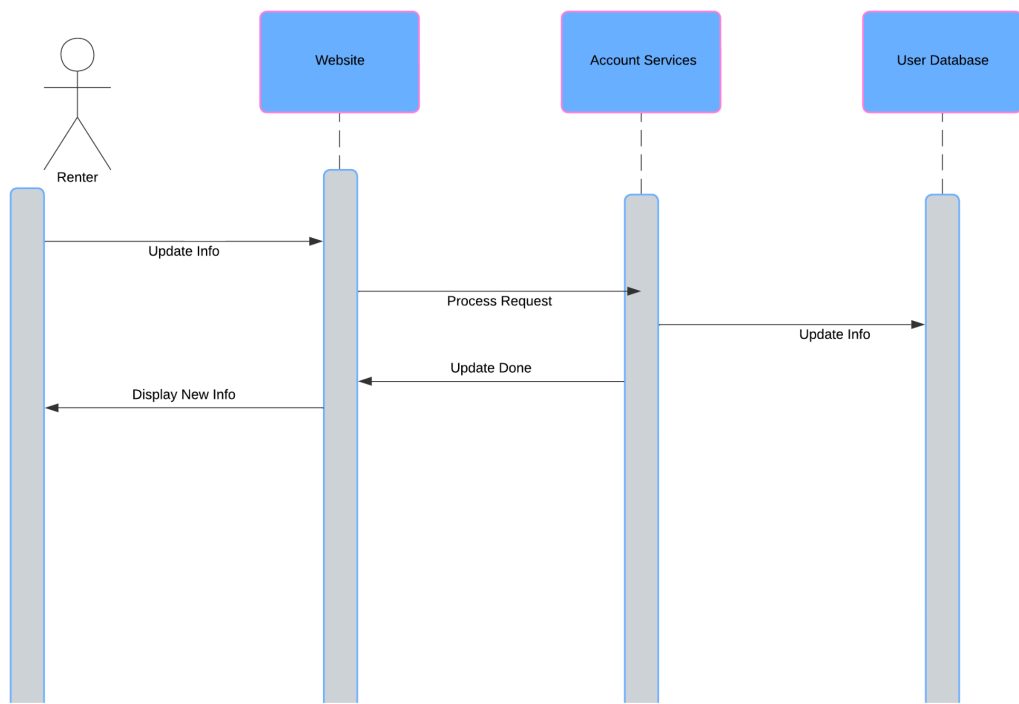
Rentee: Customer Support



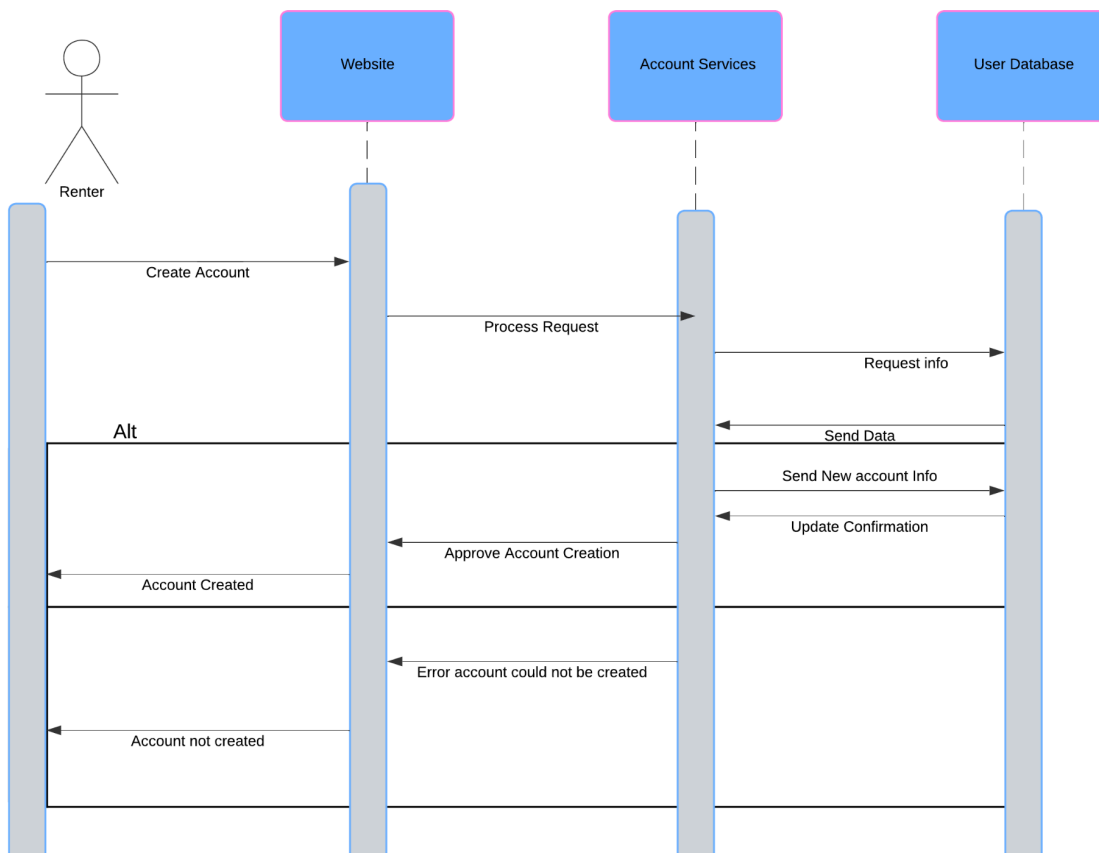
Rentee: Leave a Review



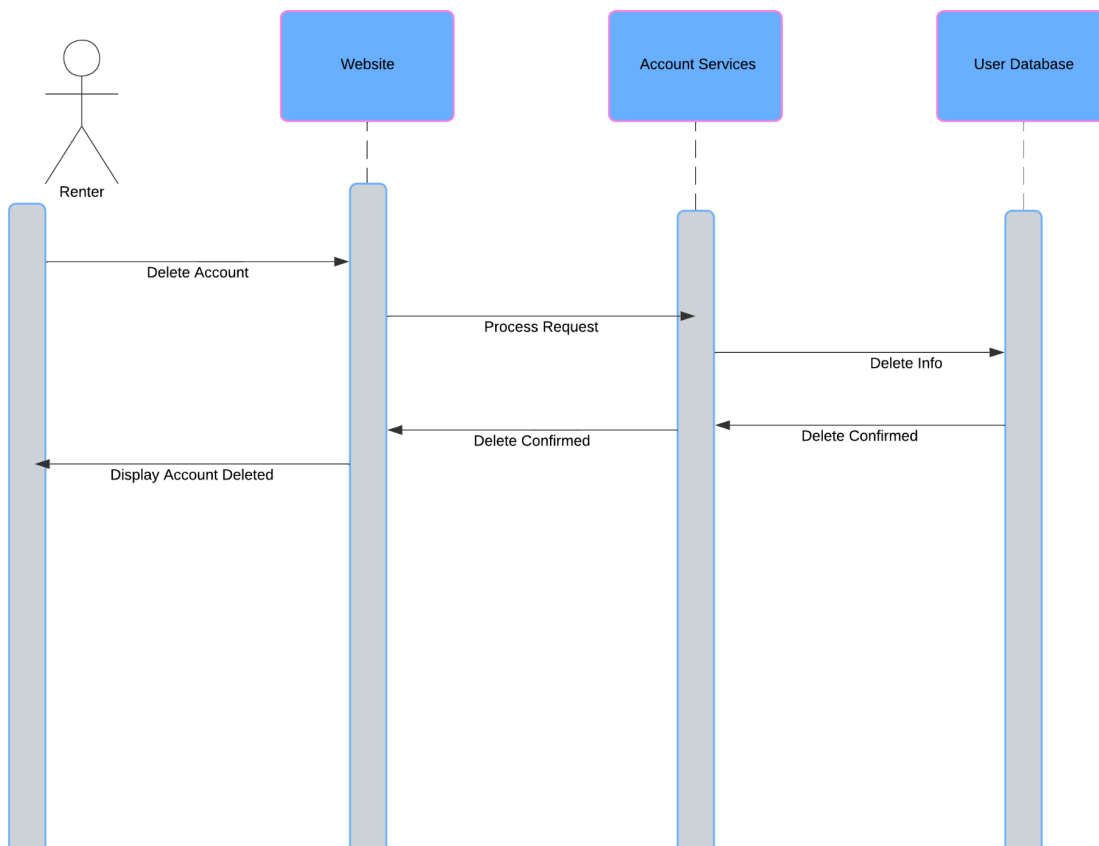
Renter: Update Account



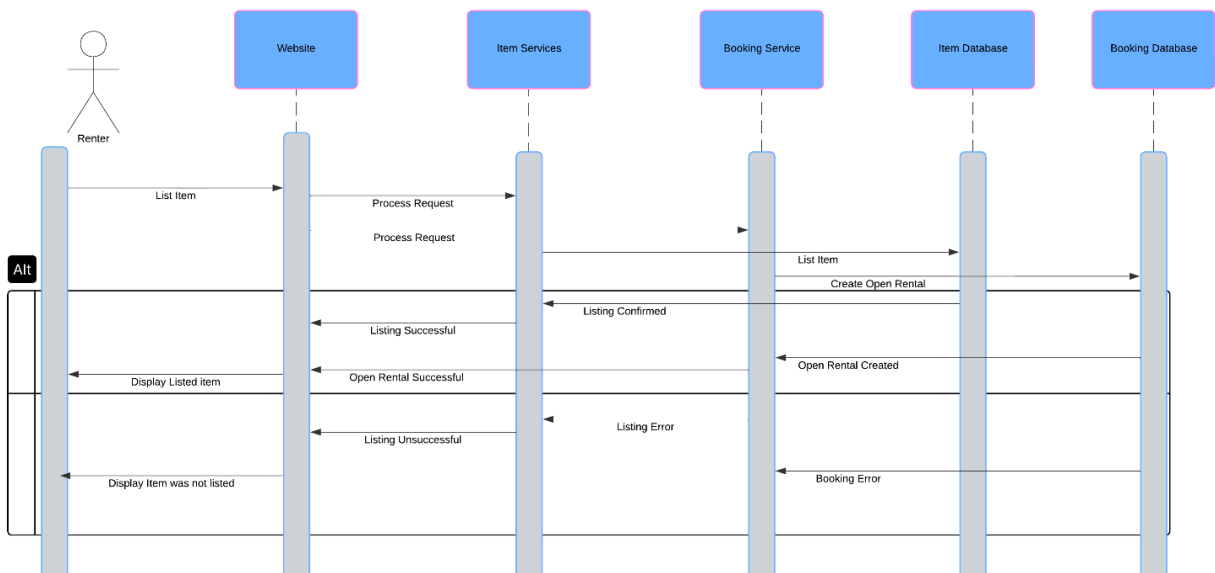
Renter: Create Account



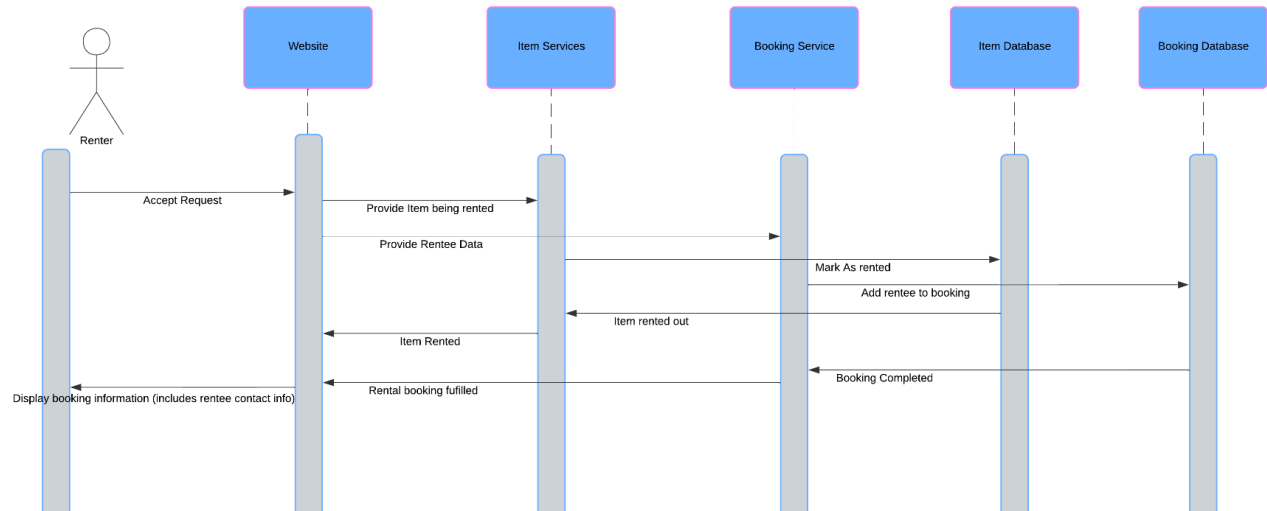
Renter: Delete Account



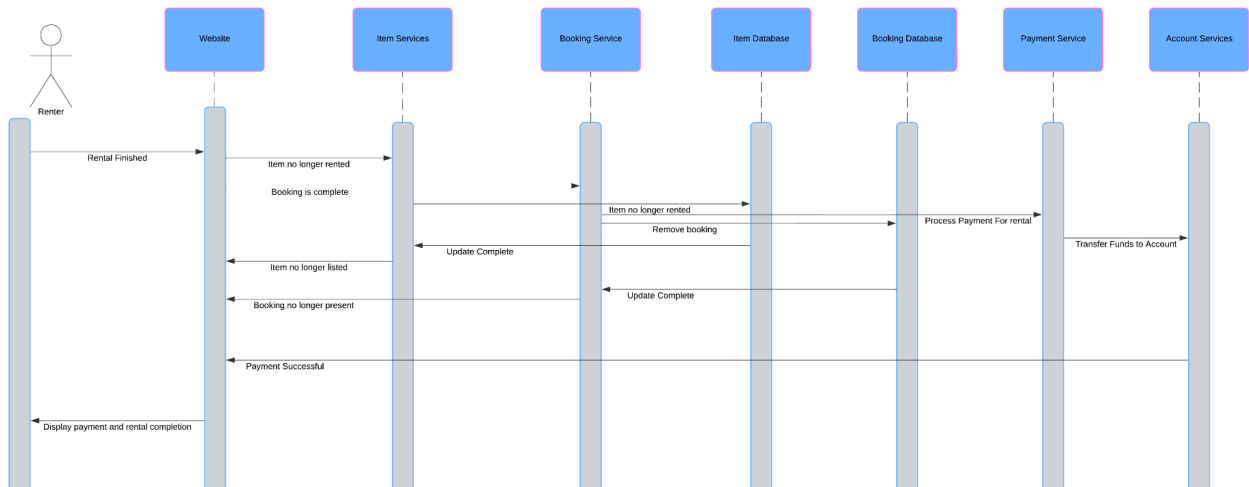
Renter: Listing Items



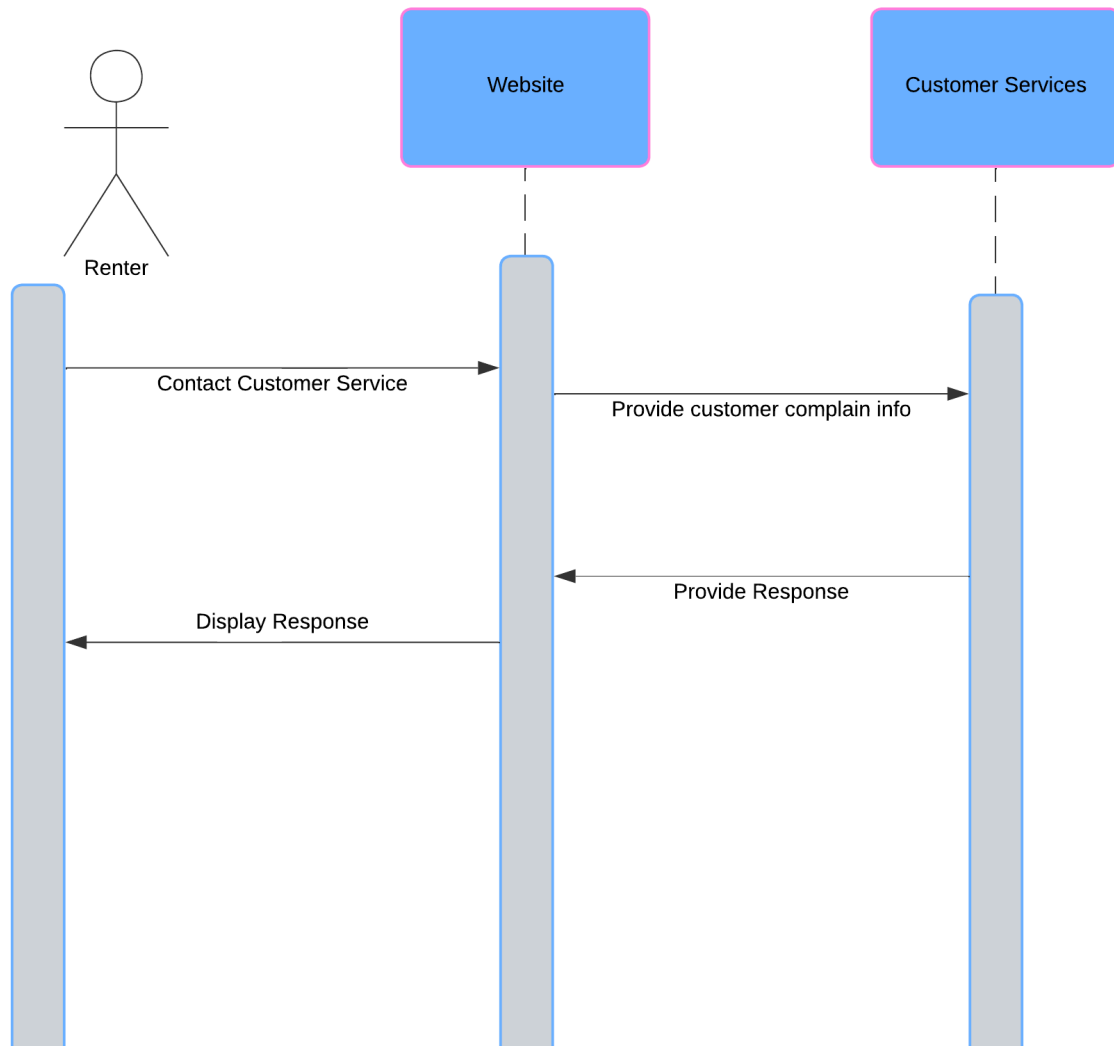
Renter: Manage Rental Request Accept



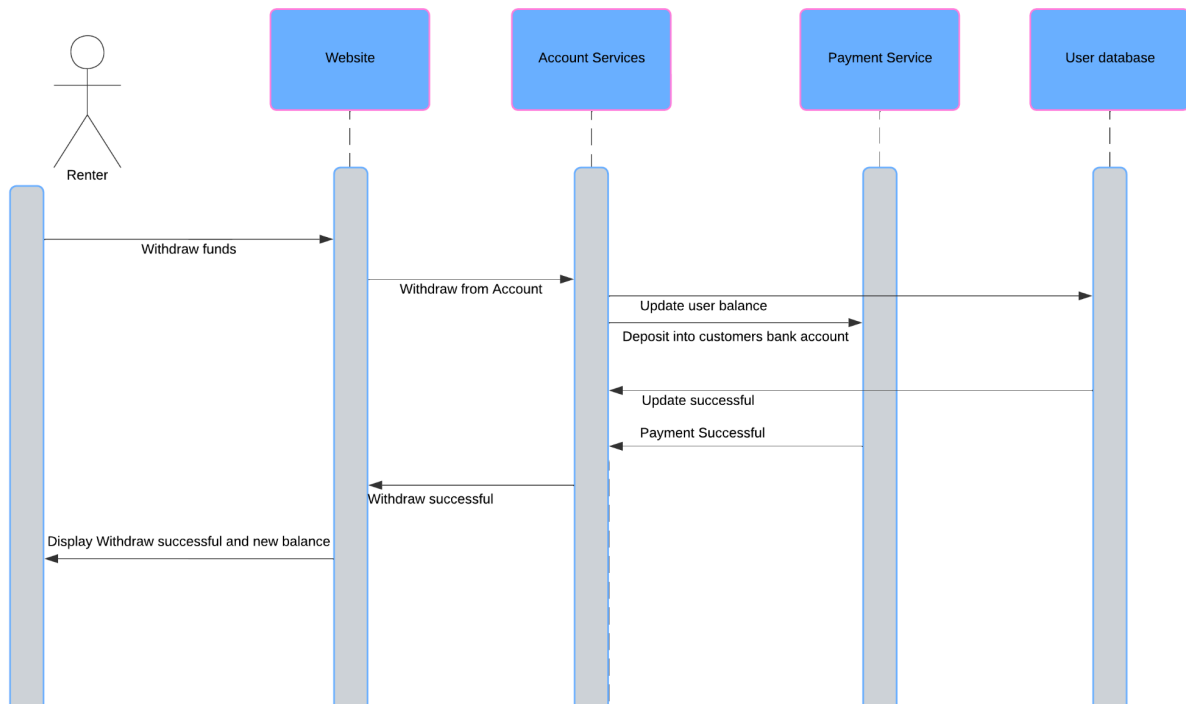
Renter: Manage Rental Request Finish Rental



Renter: Contact Customer Service

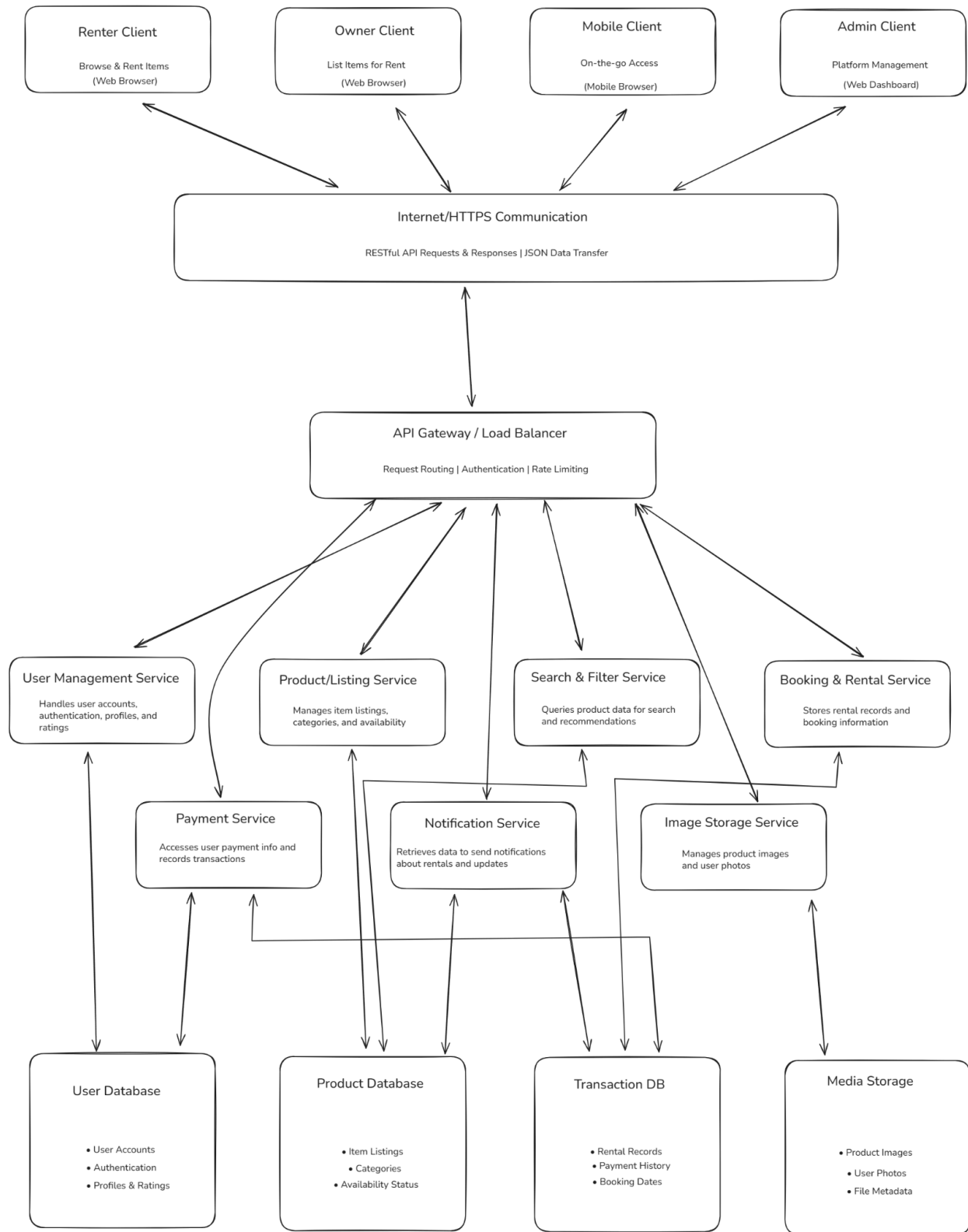


Renter: Withdraw Funds



Class Diagram





4a. Project Scheduling

Schedule

- Start Date: December 1, 2025
- End Date: June 1, 2026
- Duration ~6 months (24 weeks)

Justification:

A six-month schedule provides ample time to complete the full life cycle of the project—which consists of planning, development, integration, testing, and deployment—without rushing critical tasks. By structuring the work into incremental sprints, each phase delivers tangible progress and reduces risks, allowing early feedback and smoother integration of features.

This timeline also ensures that testing, performance tuning, and final refinements are handled thoroughly rather than compressed at the end. The built-in buffer adds flexibility for unexpected issues/events, which helps maintain quality while still meeting the target delivery date.

1. Work Policy:
 - Weekends: Not counted in the schedule (not accounted for)
2. Days and Hours Worked: 8 hours per day, Monday to Friday (9am-5pm; Full-Time)

Phase Breakdown:

- Planning & Architecture (2 weeks) - Define scope, design, and database setup
- Sprint 1 (4 weeks) - User accounts and authentication
- Sprint 2 (4 weeks) - Item listing and search features
- Sprint 3 (4 weeks) - Booking and payment integration
- Sprint 4 (3 weeks) - Reviews, support, and admin tools
- Sprint 5 (3 weeks) - Testing and performance tuning
- Sprint 6 (3 weeks) - Deployment and user testing
- Buffer (1 week) - Final fixes and rollout

4b. Cost, Effort and Pricing Estimation:

1. Using the agile estimation method found in the book using the fibonacci numbers for the story points. The assumption is a relatively inexperienced team working for \$20 / hour and that each SP will be about 20 hours of development time. [4]

User Story	Story Points (SP)	Hours
------------	-------------------	-------

Create account	5	100
Remember login	5	100
Authentication	8	160
Update profile	5	100
Delete account	5	100
Roles registration	5	100
Create listing	13	260
Update listing	5	100
Delete listing	8	160
View item details	5	100
Ask questions	3	60
View owner contact	3	60
Search	5	100
Filter	5	100
Sort	8	160
Rental request	8	160
Availability check	8	160
Approve/deny rental	5	100
Confirm return	8	160
Pay securely	5	100
Deposit hold	5	100
Payment history	5	100
Receipts	3	60
Email confirmations	3	60
Reviews	5	100

Ratings	3	60
FAQs / guidelines	3	60
Support / tickets	3	60
Notifications	3	60

Total hours of dev time = 3100

Assuming each person works 40 hours a week = 240 hrs a week

$3100/240 = 12.92$ Weeks of development time

$3100.08 * 20 = \text{\$62016 cost in development time}$

12.92 weeks of pure dev time does not account for meetings/training/error/ inexperienced dev team/ idleness so give about double time to account for all of this so total project time is 24 weeks.

20 dollars/hour * 40 hours/week = 800 dollars per week per worker

$800 * 6 = 4800$ dollars a week for all workers

$4800 * 24 = \text{\$115,200 for 24 weeks of paying for labor}$

2.Function point estimation

Assuming Average Complexity

Category	Count	Simple	Average	Complex	Count*Complexity
User Input	18	3	4	6	72
User Output	6	4	5	7	30
User Queries	12	3	4	6	48
Data Files and relational tables	6	7	10	15	60
External Interfaces	2	5	7	10	14
				GFP	224

PCA

Essential = PC 1, PC 2, PC 4, PC 6, PC 8

Everything else is moderate

$$PCA = .65 + .43 = 1.08$$

$$FP = 224 * 1.08 = 241.92 \text{ FP}$$

Assume that productivity is 5 functions points per person- week

$$E = 241.92 / 5 = 48.384 \text{ Weeks}$$

$$48.384 / 6 = 8.064 \text{ weeks for project duration}$$

$$6 * 40 = 240 \text{ hours a week}$$

$$240 * 8.064 = 1,935.36 \text{ hours}$$

$$1,935.36 * 20 = \$38,707.20 \text{ cost for labor for project}$$

For development time

4c. Cost of Hardware Products

Macbooks \$750/per person = \$4500

4d. Cost of software

PostgreSQL = 0.00 (Open Source)

Servers: AWS EC2 \$75/month = \$450

Database: AWS RDS \$50/month = \$300

Storage: AWS S3 100GB \$25/month = \$150

4e. Cost of personnel

\$115,200 for 24 weeks of labor

Total Cost for 24 Week project = \$120,600

5. Testing Plan

Responding to a ticket

- Test that the user's status is updated correctly when a ticket is responded to.
- **Interaction:** Ticket class to User class
- **Method:** updateStatus(staffMessage, status)

- **Input:** The new status of the ticket and staff response.
- **Behavior:** The function is intended for staff to use and will update the status of the ticket, write the staff's response, and send a notification to the user's notification list.
- **Positive test:** When a ticket's status is updated, its status will be updated to the given status entered by the staff, the staff response will be updated to match staffMessage, and the associated user will have one new notification appended to their notification list.
- **Negative test:** The test will fail if the status or staff message are not equal to their respective inputs or if the notification list of the associated user does not have only 1 more notification.

Positive Test Cases

- updateStatusTest(): positive tests only
 - **Results:** Pass; status and staff message matches input and there is $0 + 1 = 1$ notification for the user.

Negative Test Cases

- updateStatusNull(): test if status or staff message are ever null
 - **Results:** Pass; status and staff messages are never null
- updateStatusEmptyString(): test if status or staff message are ever empty strings
 - **Results:** Pass; status and staff messages are never empty strings
- updateStatusOneCharacter(): test if status or staff message are ever one character
 - **Results:** Pass; status and staff messages should never just be one character anyways.
- updateStatusWrongString(): test status and staff message against incorrect strings
 - **Results:** Pass; status and staff messages are only equal to their inputs and nothing else.
- updateStatusWrongNotifCount(): tests if notification count is ever not incremented by only 1
 - **Results:** Pass; notification list only has one more notification

6. Comparison of your work with similar designs

1. Fat Llama

Overview: Fat Llama is a rental marketplace that allows users to rent out almost anything they would want, from cameras and drones to furniture and party supplies. It includes insurance coverage, user reviews, and secure payment handling for transactions. [1]

Comparison:

- Similarities: Like Fat Llama, Rent Impromptu enables individuals to list and rent personal items.
- Comparison: Fat Llama operates as a platform with service fees and centralized insurance. Our Rent Impromptu focuses on local, low-friction, community-based exchanges, targeting students or users who may not need insurance for lower value items. Our system prioritizes simplicity, minimal cost, and ease of use over comprehensive commercial features.

2. Turo Overview: Turo specializes in car rentals between private owners and renters. It includes identity verification, vehicle inspection, and dynamic pricing. [2]

Comparison:

- Comparison: Turo is vehicles only, whereas Rent Impromptu is more general-purpose, supporting a wider variety of item categories.
- Similarities: Turo emphasizes high-stakes rentals requiring regulatory compliance. Rent Impromptu assumes lower-risk, short-term rentals among trusted community members, reducing the need for heavy verification layers in the MVP, and instead as a future implementation.

3. Peerby

Overview: Peerby was a neighborhood-based sharing app that allowed users to borrow and lend items for a small fee. [3]

Comparison:

- Similarities: Rent Impromptu has some similarities from Peerby's community-centric application.
- Comparisons: Unlike Peerby, which lacked robust search and categorization, Rent Impromptu will implement smart categorization of listed items (e.g., "Outdoor Gear," "Electronics," "Study Tools") and search filters to enhance discoverability—especially useful in dense urban or campus environments.

7. Conclusion

Rent Impromptu addresses the need in communities to share and rent all types of items, which helps reduce financial and storage constraints. We have created a six-month development outline that will allow us to achieve goals of functionality, usability, and cost-effectiveness in our platform. Our estimated cost is around 120,600 USD, enabling our client-server architecture to deliver a secure and maintainable solution through PostgreSQL and AWS cloud services. Core features like user account management, item listing and search, rental reservations, payment processing, and customer support create an efficient environment for peer-to-peer item sharing.

We pride ourselves on our focus on simplicity and community-based exchanges, which differentiates us from competitors like Fat Llama, Turo, and Peerby. We aim to target local users, specifically students and local community members, which allows us to eliminate unneeded complexity while maintaining features like ratings, reviews, and secure payments. Our categorization and search features address gaps found in previous platforms, making it more intuitive and accessible for our users. We aim to complete this project through a structured development approach, including planning, five development sprints, testing, and deployment to ensure quality deliverables and allow iterative improvements, and we will include testing and validation within those cycles.

Rent Impromptu aims to build a community by fostering sustainable consumption patterns. As we are currently working towards deployment, we will continue working to create a platform that serves our community's needs while also spreading environmental awareness.

8. References

- [1] Fat Llama, “How It Works,” *Fat Llama*, 2025. [Online]. Available: <https://www.fatllama.com>
- [2] Turo, “Rent a Car from a Local Host,” *Turo*, 2025. [Online]. Available: <https://www.turo.com>
- [3] Peerby, “Borrow and Lend Locally,” *Peerby*, 2023. [Online]. Available: <https://www.peerby.com>
- [4] “Mapping story points with hours,” *Scrum.org*, 2024. [Online]. Available: <https://www.scrum.org/forum/scrum-forum/8360/mapping-story-points-hours>